ENSF 612 Project Proposal (Milestone 1, Revised)

Engineering Large-Scale Data Analytics Systems, Fall 2026
Instructor: Dr. Ebrahim Karami
October 26, 2026

1 Project Title

DupHunter: Scalable Duplicate Question Detection on Stack Overflow

2 Group Members

Name	Main role
Satyam Bhanot	Data pipeline, Spark cluster, scaling study (E8)
Iesrat Jahan	Text representation, MinHash, LSH and its settings (E1, E2, E3)
Md Jubayer Khan	Answer key, evaluation, baselines, SON tag blocking (E5, E6)
Navjot Singh Shienh	Checking and ranking candidates, handling oversized LSH buckets (E4, E7)

3 Problem Statement

Stack Overflow is a website where programmers ask and answer questions. Many new questions have already been asked before, just in different words. These are called *duplicates*. Today, volunteers find and close them by hand, and many are missed.

We want to build a system that finds duplicate questions automatically. The hard part is size. Stack Overflow has about 24 million questions, and comparing every question with every other one would take years on one computer. We need methods built for large data that can run on more than one machine.

One problem, studied in depth. After feedback on our first proposal, we dropped the AI-impact study and the streaming check, which were only loosely tied to duplicate detection. We use that time to go deeper: each stage of the system is a design choice settled by an experiment (E1 to E8, Table 2).

Questions we want to answer:

Q1. **(Core)** How many known duplicates can our MinHash and LSH system find, and how close does it come to an exact search that checks every pair?

Q2. **(Core)** Do MinHash and LSH behave the way the course theory predicts on real questions, and which settings give the best balance between missed duplicates and wasted work?

Q3. **(Core)** What slows the system down as it grows (oversized buckets, data moved between machines, memory), and how does run time change with more cores, a second machine, and more data?

Q4. Does grouping questions by frequent tag sets, found with SON, save work without losing many duplicates?

Expected outcome: a working Spark system for duplicate search, and eight experiments that explain each design choice and its cost.

4 Motivation and Application

Duplicate questions waste time. People wait for answers that already exist, experts answer the same thing again, and volunteers spend hours searching. A tool that finds duplicates could show a user an existing answer before they post. The same kind of search is used to remove repeated pages from web search, to catch copied documents, and to clean AI training data.

At this size, a good algorithm is not enough: the data must be spread over machines, common phrases can overload one machine, and every setting trades accuracy against work. This is what ENSF 612 teaches: MapReduce and Spark, shingling, MinHash, LSH, and frequent itemsets [1].

5 Dataset

Source. We will use the public Stack Exchange data dump on the Internet Archive (`https://archive.org/details/stackexchange`). The newest release is from September 30, 2026 [2]. It is free to use under the CC BY-SA 4.0 license.

Size and format. The Stack Overflow archive is a single 64.2 GiB (68.9 GB) compressed file (`.7z`) covering July 2008 to September 2026. Inside, each table is an XML file with one record per line. We only need the columns in Table 1 and skip every other table. Post counts are live totals from the Stack Exchange API (October 28, 2026); the September dump is slightly smaller.

Table 1: The data we will use.

Table	Records	Fields we keep	What we use it for
Posts	60.3 million (24.2 million questions)	id, type, title, body, tags, date	Question text, tags, and dates
PostLinks	about 10 million (counted on loading)	post id, related post id, link type	Known duplicate pairs (our answer key)

How much data we will use.

- Our main runs use the 2.2 million questions tagged `python`. This is still large: they make about 2.4 trillion pairs. We only score duplicate pairs where both questions are in the subset.
- If time allows, we run the final system on all 24 million questions to see how it behaves on data more than ten times larger.
- We build and debug everything on the Super User site first (517,000 questions, 1.4 GB compressed, same format, about 47 times smaller).

Data quality and limitations.

- Question text is stored as HTML, with code mixed into the writing.
- Deleted posts are missing.
- Not every duplicate was marked by volunteers, so our answer key is incomplete and our precision will be a lower bound.
- Some duplicates share few words with their original. We first measure how similar known duplicates are compared with random pairs (E1).
- Results on one tag may not hold for every topic.

Preprocessing. We extract only the two tables and read them as a stream, so we never store hundreds of GB of unpacked XML. We keep the needed columns and save them as Parquet (a compressed table format that Spark reads quickly), split by year. For each question, we remove HTML tags, keep code blocks separate, and break the text into short groups of words called *shingles*.

6 Proposed Methodology

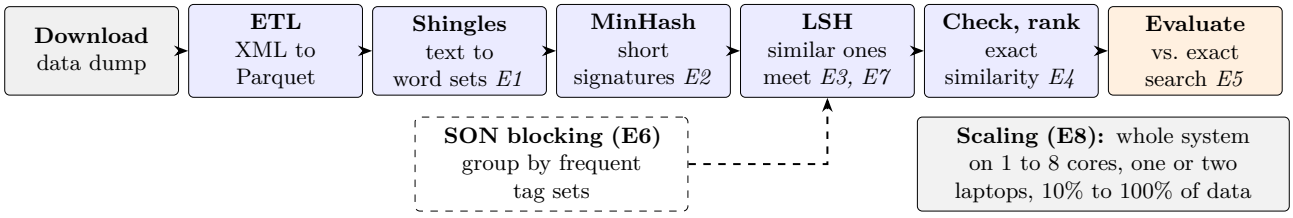


Figure 1: DupHunter. Each stage is a design choice tested by the experiment named in it. The dashed box is the secondary part.

The core. We turn each question into a set of shingles and compress it into a short MinHash *signature*. LSH cuts each signature into bands; questions that match on a whole band land in the same bucket, and we only compare questions that share a bucket. We then check each candidate pair with its exact similarity and rank the candidates for each question. All of these are our own Spark code, written in the MapReduce style [3]. Checking every pair grows with the square of the number of questions; MinHash and LSH grow roughly in line with it, plus the candidates they produce, which we keep on disk and count in every run.

How we will evaluate. Accuracy uses Recall@K and Precision@K (K = 1, 5, 10), MAP, and NDCG (how high the real duplicate is ranked), plus the reduction ratio (the share of pairs we never compare). We also break recall down by similarity, tag, question length, and year, to see where and why the system fails. For speed we record run time, speedup (how many times faster than one core), data moved between machines, peak memory, and the time of the slowest task. Each scaling setting runs

Table 2: Experiments. Each one settles a design choice or measures a cost.

Exp.	Stage	What we compare	What we measure	Q
E1	Shingles	Word groups of 1 to 5 words vs. character groups; title only vs. title and body; with and without very common shingles (boilerplate code)	How well real duplicates stand apart from random pairs	Q1
E2	MinHash	Signatures of 32 to 256 numbers	Error of the similarity estimate vs. the error theory predicts; time and memory	Q2
E3	LSH	Several band and row settings	Measured S-curve vs. the one theory predicts; recall vs. number of candidates	Q1, Q2
E4	Check and rank	Ranking by signature alone vs. exact similarity vs. TF-IDF re-ranking	Recall@K, Precision@K, MAP, NDCG	Q1
E5	Baselines	Exact search of every pair on a sample of 10,000 to 100,000 questions; Spark's built-in `MinHashLSH`	Accuracy given up for the time saved	Q1
E6	Blocking	No blocking vs. single tags vs. SON frequent tag sets	Run time, data moved, recall lost	Q4
E7	Big buckets	Size limit vs. splitting big buckets over many tasks vs. dropping common shingles	Time of the slowest task, recall, memory	Q3
E8	Scaling	Table 3	Run time, speedup, data moved, peak memory	Q3

three times.

Compute setup. We have two laptops with 16 GB of RAM each. Laptop A has an Intel Core i7-13650HX (14 cores) and Laptop B an AMD Ryzen 7 5800H (8 cores). Both run Windows, so Spark and HDFS run inside WSL2 (Linux on Windows). We connect them as a Spark standalone cluster [4] on the same wired network: Laptop A runs the master, the driver, and one worker; Laptop B runs one worker. Each worker gives Spark 8 cores and about 8 GB of memory (7 GB on Laptop B, which has 13.9 GB usable). The data is stored in HDFS [8] with a copy on each laptop, so every worker reads from its own disk. Spark runs tasks inside *executors*, its worker processes. For the 1, 2, 4, and 8 worker tests we give one executor 1, 2, 4, and then 8 cores, because each extra executor needs its own memory and eight would not fit in 16 GB. MinHash signatures for 2.2 million questions need only about 1 GB.

Table 3: Scaling tests (E8) on the two-laptop cluster.

Test	Setup	What it shows
A. More cores	1, 2, 4, and 8 cores on Laptop B, whose 8 cores are all the same; same data	Speedup, and how much of the work cannot run in parallel
B. Second machine	8 cores on A alone, on B alone, 4 + 4 on both, then 8 + 8	The cost of the network, and how the slower laptop holds back the faster one
C. More data	10%, 25%, 50%, and 100% of the subset; all of Stack Overflow if time allows	How run time and memory grow with data size

Priority and cut order. Q1 to Q3 must be finished. If time runs short, we cut, in this order: the run on all of Stack Overflow, splitting big buckets (we keep the simple size limit), TF-IDF re-ranking, and SON blocking (Q4). Even after these cuts, every member still has core results. The experiments are cheaper than they look: E3, E6, and E7 rerun one pipeline with one setting changed, and E1 and E2 use only a few thousand pairs.

7 Algorithms and Techniques

We write the course algorithms ourselves. Exact search on a sample, Spark's `MinHashLSH`, and Spark's `FPGrowth` are used only for comparison, so the report clearly shows which code is ours.

- **Core, our own code:** shingling with removal of very common shingles (E1); MinHash [5] (E2);

LSH banding [6] (E3); exact checking and ranking of candidates (E4); handling of big buckets [9] (E7); all as Spark jobs in the MapReduce style [3] (E8).

- **Secondary:** SON frequent itemsets [7] for tag blocking (E6).
- **Stretch:** TF-IDF re-ranking [1] (E4), splitting big buckets over many tasks (E7), and the run on all of Stack Overflow (E8).

8 Expected Challenges

Challenge	How we plan to handle it
Duplicates written in very different words	Measure it first (E1); report recall by similarity level
Missing duplicate labels	Treat precision as a lower bound
Huge buckets (common phrases) slow one task	Measure the slowest task; compare a size limit with splitting buckets (E7)
Network traffic; 16 GB per laptop	Wired network; process bands in batches
Spark and HDFS on Windows (WSL2)	Set up in the first week; test with a small job early; fall back to port forwarding
Two different laptops (Intel and AMD)	Time each one alone first (test B)
Blocking misses some duplicates	Measure the recall lost compared with no blocking (E6)

9 Expected Results

- **Q1:** LSH will find most known duplicates while skipping more than 99.9% of all pairs; exact checking will rank the real duplicate higher, close to Spark's built-in version.
- **Q2:** measured results will follow the theory closely; 128 numbers per signature will be enough.
- **Q3:** MinHash will speed up almost in line with the cores, as it needs no communication. LSH will gain less, because data moves between machines and big buckets slow it; splitting them will help.
- **Q4:** tag blocking will cut run time and data moved, with a small loss in recall.

10 Initial Project Schedule

The core comes first. Midterm week (October 30 to November 3) is kept light: only downloads, the ETL, and cluster setup. On November 13 we check progress and decide which stretch parts go ahead.

Table 4: Weekly plan (blue: main work; orange: milestone).

Task (week starting)	Who	Oct 26	Nov 2	Nov 9	Nov 16	Nov 23	Nov 30
Midterm (light week)	All	Oct 30	Nov 3				
Download, ETL, cluster setup	Satyam						
Shingles, MinHash, LSH (E1, E2, E3)	Iesrat						
Answer key, evaluation, baselines (E5)	Jubayer						
Checking candidates, big buckets (E4, E7)	Navjot						
Core checkpoint	All			Nov 13			
Scaling study (E8)	Satyam						
SON tag blocking (E6)	Jubayer						
Results freeze	All				Nov 20		
Presentation	All						
Final report and code	All						Dec 4

11 References

[1] A. Rajaraman and J. D. Ullman, *Mining of Massive Datasets*. Cambridge University Press, 2011.

[2] Stack Exchange, Inc., "Stack Exchange Data Dump 2026-09-30," Internet Archive, https://archive.org/details/stackexchange_20260930.

[3] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," in *Proc. OSDI*, 2004.

[4] M. Zaharia *et al.*, "Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing," in *Proc. NSDI*, 2012.

[5] A. Z. Broder, "On the resemblance and containment of documents," in *Proc. Compression and Complexity of Sequences*, 1997.

[6] P. Indyk and R. Motwani, "Approximate nearest neighbors: Towards removing the curse of dimensionality," in *Proc. ACM STOC*, 1998.

[7] A. Savasere, E. Omiecinski, and S. Navathe, "An efficient algorithm for mining association rules in large databases," in *Proc. VLDB*, 1995.

[8] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, "The Hadoop distributed file system," in *Proc. IEEE MSST*, 2010.

[9] Y. Kwon, M. Balazinska, B. Howe, and J. Rolia, "SkewTune: Mitigating skew in MapReduce applications," in *Proc. ACM SIGMOD*, 2012.